전자전기컴퓨터설계실험II

실험 전 보고서

LAB06 UART 통신과 FPGA 주변장치

서보모터 · SRAM · DAC · STM32 UART 과제 5·6·7

이름	조하늘
학번	2022750030
작성일	2026년 10월 21일
실험 예정일	2026년 10월 26일
검증 환경	VS Code / Icarus Verilog / VaporView
대상 장비	Combo II-DLD / NUCLEO-G474RE
보고서 범위	FPGA 사전 시뮬레이션과 STM32 UART 실험 준비

보고서 범위
제공된 파형·빌드 화면 11장과 LAB06 폴더의 소스·테스트벤치·저장된 실행 로그를 대조하였다. 보드 실측 및 통신 시연은 실험 수행 계획으로 구분한다.

실습 범위는 최신 제공 자료인 STM32 실습 안내의 10월 26일 일정에 따른다[2].

1 목적과 검증 조건 및 서보모터 제어

실험 목적과 근거

PC에서 보낸 문자를 STM32와 FPGA를 거쳐 전달하고, 문자 명령으로 LED를 제어한다. FPGA 교재 실습은 서보 위치 제어, SRAM 읽기·쓰기, 병렬 DAC 출력이다[2].

파형 사진은 「lab06 사전보고서 사진.docx」에서 추출했다. 사전 검증 판정은 기존 simulation.log와 TB를 대조한 결과이며, 이번 보고서 작성 중 새로 실행한 결과는 아니다[4].

서보 구조와 핵심 코드

servo_position이 펄스 폭을 100 μs씩 조정하고 servo_pwm이 프레임 경계에서 폭을 반영한다. 카운터와 상태 레지스터로 주기를 만드는 순차회로 구조이다[1].

```
FRAME_CYCLES = CLK_HZ / 50;
if(count == FRAME_CYCLES-1) begin
count <= 0;
width_cycles <= next_width;
end
assign servo = !rst_p &&
(count < width_cycles);
```

보드 최상위는 버튼 동기화·디바운스 후 한 번 누름을 한 클록 펄스로 변환한다. 좌우 버튼 동시 입력은 위치를 유지한다.

계산값과 시뮬레이션 비교

조건	보드 HIGH	TB 클록 수
최소	1.0 ms	100
리셋	1.5 ms	150
최대	2.0 ms	200
프레임	20 ms	2,000

보드는 50 MHz이므로 프레임 1,000,000클록, HIGH 50,000·75,000·100,000클록이다. TB는 CLK_HZ=100,000으로 계수값을 줄이지만 실제 입력 클록은 10 ns이다. 따라서 파형상 프레임은 20 μs, HIGH는 1·1.5·2 μs이다.

그림 1의 0x05DC→0x03E8→0x07D0은 1500→1000→2000 μs의 설정값이다. 그림 2의 초기 HIGH 약 1500 ns는 TB의 150클록과 일치한다.

저장 로그는 최소·최대 포화, 동시 입력, 재리셋 검사를 PASS로 기록한다. 종료 시각은 100,036 ns이다. 로그 문구의 "20 ms"는 보드 환산값이다.

보드 확인 계획

키패드 4·6번으로 위치를 바꾸고 AA22 출력의 50 Hz와 세 펄스 폭을 측정한다. 펄스 폭과 회전각의 실제 대응은 실측한다.

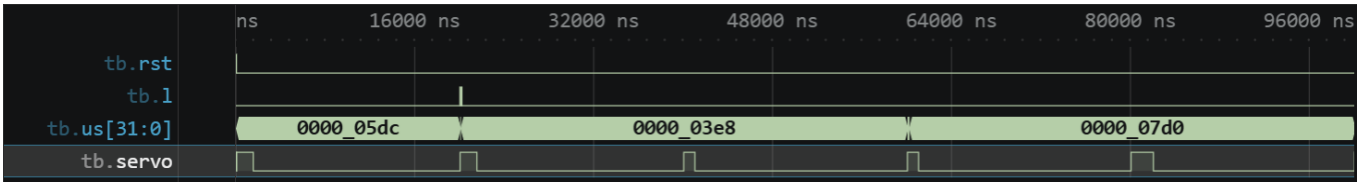


그림 1. 서보 설정값의 최소·최대 전이와 주기 출력. 원본의 파형 부분을 발췌했다.

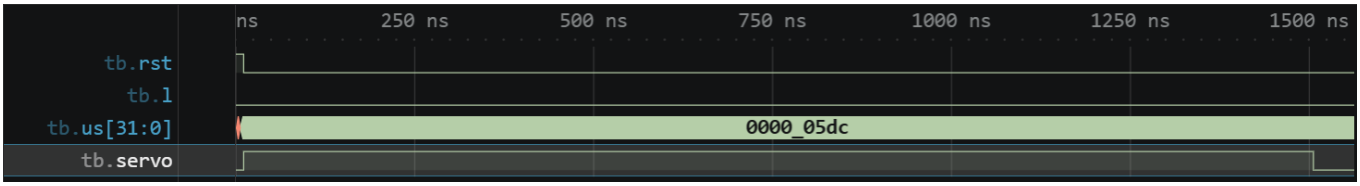


그림 2. 초기 설정 1500에서 HIGH 구간을 확대한 파형. 보드의 1.5 ms와 TB의 1500 ns를 구분한다.

2 FPGA SRAM 읽기와 쓰기

설계 목표와 버스 제어

주소별로 값을 저장한 뒤 같은 주소에서 읽어 LED에 표시한다. SRAM은 주소로 저장 위치를 선택하는 메모리이며 읽기와 쓰기에 따라 버스 구동 주체가 달라진다[1].

```
drive_data = ctrl_w && !ctrl_r;
read_data = ctrl_r && !ctrl_w;
ram_cs = !(drive_data || read_data);
ram_oe = !read_data;
ram_we = !drive_data;
```

읽기 시 FPGA는 데이터 버스를 Z로 해제하고 SRAM 모델이 구동한다. 대기 및 동시 요청 시에는 CS·OE·WE를 모두 HIGH로 하여 버스 충돌을 방지한다[4].

요청 W/R	CS/OE/WE	버스
0 / 0	1 / 1 / 1	Z
1 / 0	0 / 1 / 0	FPGA 구동
0 / 1	0 / 0 / 1	SRAM 구동
1 / 1	1 / 1 / 1	Z

입력 조건과 확인된 결과

TB는 주소 i=0~15에 15-i를 쓴 뒤 같은 주소 순서로 읽는다. 쓰기 구간에서는 OE가 HIGH, 읽기 구간에서는 WE가 HIGH여야 한다. 대기 시 LED는 0이다.

그림 3에서 주소 0~F에 F~0을 쓰고 다시 0~F를 읽는다. 그림 4의 주소 C·D·E·F에서 버스 값은 0003·0002·0001·0000이며 LED도 03·02·01·00으로 대응한다.

읽기 사이의 ZZZZ는 버스 해제 상태다. 마지막 동시 요청에서도 제어 신호는 모두 HIGH이고 버스는 Z이다. 저장 로그는 16개 주소, 버스 해제, 동시 요청 검사를 PASS로 기록하며 164 ns에 끝난다.

보드 시험과 한계

DIP SW1~4로 주소, SW5~8로 데이터를 선택하고 키패드 2번으로 쓰기, 3번으로 읽기를 수행한다. 주소 3에 1010을 기록한 뒤 다른 주소를 거쳐 돌아와 저장값을 확인한다.

TB의 SRAM 모델은 실제 칩의 접근 시간과 전기적 특성을 재현하지 않는다. 실물에서는 접근 시간·WE 폭·버스 방향을 확인한다. 전원 재인가 후 값은 보장되지 않으며 반드시 0이 된다고 판단하지 않는다.

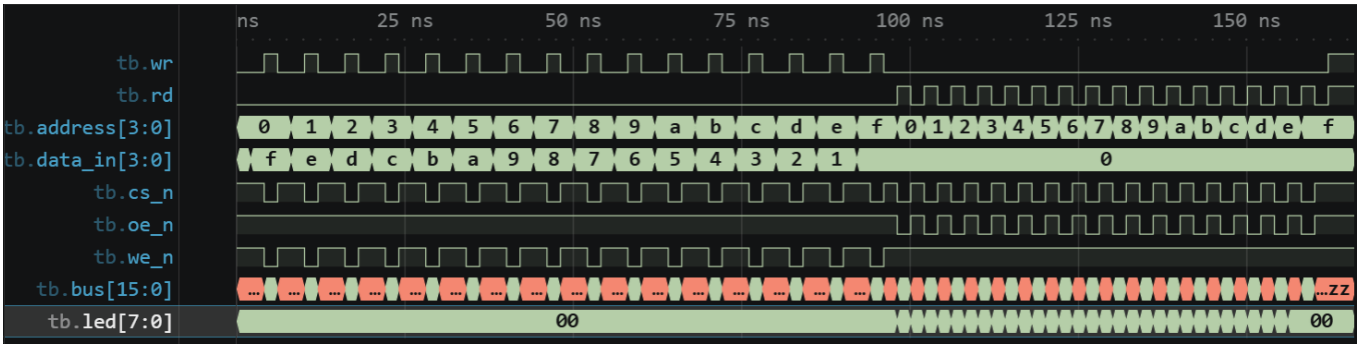


그림 3. 16개 주소의 쓰기·읽기 전체 파형. 앞부분은 쓰기, 뒷부분은 읽기이다.

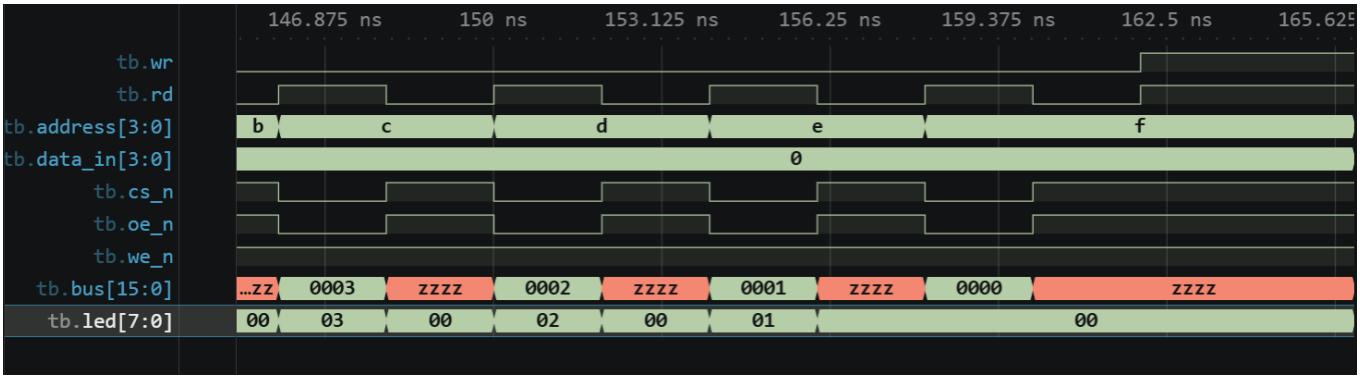


그림 4. 주소 C~F 읽기와 마지막 동시 요청 구간. Z는 고임피던스 상태이다.

3 FPGA 병렬 DAC 출력

설계 구조와 예상 출력

ramp8이 일정 간격으로 8비트 코드를 증가시키고 dac_parallel8이 데이터·채널과 LOW 활성 쓰기 신호를 출력한다. CS와 LDAC은 LOW로 고정되어 있다[4].

```
we_n <= 1;
if(update) begin
data <= value;
a_b <= channel;
we_n <= 0;
end
```

보드에서는 UPDATE_CYCLES=500,000이고 클록이 50 MHz이므로 단계 간격은 10 ms이다. 256단계가 반복되어 예상 톱니파 주기는 2.56 s이다.

항목	보드	TB
갱신 간격	10 ms	40 ns
한 바퀴	2.56 s	10.24 µs
WR LOW	20 ns	10 ns

단극성 이상적 8비트 DAC를 가정하면 Vout Vref×code/256이다. 실제 전압과 극성은 보드의 DAC 칩과 출력 증폭 회로에 따라 확인한다.

파형과 자기검사 결과

TB는 UPDATE_CYCLES=4, 입력 클록 10 ns로 260회 쓰기를 검사한다. 코드 255→0 순환, 130회 쓰기 후 채널 변경, LOW 구간의 데이터 유지와 리셋이 검사 대상이다.

그림 5에서 channel 변경 후 다음 쓰기 때 a_b가 갱신된다. 그림 6은 코드 0x11→0x12→0x13→0x14→0x15의 증가와 data·led의 일치를 보여 준다.

저장 로그의 결과는 PASS이며 종료 시각은 10,436 ns이다. TB는 WR 하강 1 ns 뒤에 시각을 저장하므로 상승 에지까지 9 ns를 비교한다. 실제 LOW 폭이 9 ns라는 의미는 아니다.

실물에서 확인할 항목

DIP SW1로 출력 채널을 선택하고 키패드 1번으로 리셋한다. 오실로스코프로 단계 간격 10 ms와 반복 주기 약 2.56 s를 측정한다. 멀티미터로 최소·최대 전압도 기록한다.

디지털 시뮬레이션은 아날로그 전압·정착시간을 검증하지 않는다. WR 20 ns가 칩의 최소 펄스 폭을 만족하는지 확인하고, 필요하면 펄스를 연장한 뒤 재검증한다.

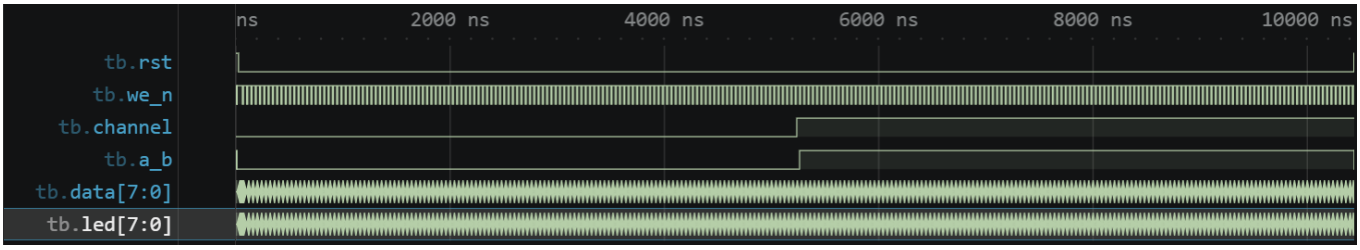


그림 5. 주기적인 쓰기 신호와 채널 변경. 전체 구간에서는 데이터 변화가 조밀하게 보인다.

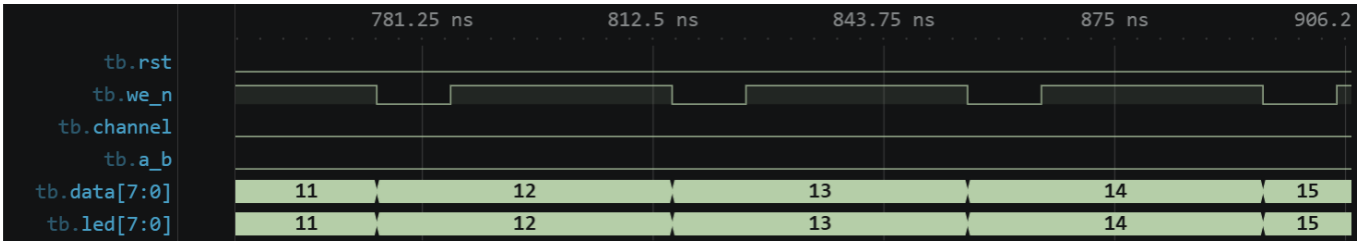


그림 6. DAC 상세 파형. 40 ns 간격의 코드 증가와 10 ns 쓰기 LOW 구간이다.

4 FPGA UART 수신과 LED 명령 처리

통신 구조와 시간 조건

uart_rx가 MCU 바이트를 받아 uart_tx를 통해 PC로 전달한다. 수신 문자를 해석하여 cmd_led를 갱신하고 나머지 7비트에는 수신값을 표시한다[4].

보드는 9600 baud, 8N1을 사용한다. 유휴 HIGH에서 시작 LOW 1비트, LSB부터 데이터 8비트, 정지 HIGH 1비트를 보낸다. 한 비트는 약 104.17 μs, 한 프레임은 약 1.042 ms이다.

```
case(rx_data)
8'h30: cmd_led <= 1'b0;
8'h31: cmd_led <= 1'b1;
8'h32: cmd_led <= ~cmd_led;
default: cmd_led <= cmd_led;
endcase
assign led = {cmd_led,last_data};
```

50 MHz에서 DIV=5208이다. TB는 DIV=10과 실제 10 ns 클록을 사용하므로 1비트가 100 ns이다. TB의 CLK_HZ=1000·BAUD=100은 비율 설정이며 파형 시간축이 실제 100 baud라는 뜻은 아니다.

입력과 예상 LED 값

순서	문자	LED1	led[7:0]
1	A	0	41
2·3	1, 1	1, 1	B1, B1
4·5	2, 2	0, 1	32, B2
6	x	1	F8
7·8	0, 0	0, 0	30, 30

그림 7·8에서 반복된 0x31은 LED1을 계속 켜고, 0x32는 매 수신마다 반전한다. 0x78은 LED1을 유지한다. 문자 "1"은 숫자 1인 0x01과 다르다.

TB는 8바이트의 PC 에코 값, LED1, 하위 7비트와 리셋을 검사한다. 로그는 PASS, 종료 시각은 16,465 ns이다.

검증 범위와 추가 시험

현재 TB는 각 에코가 끝난 뒤 다음 문자를 보낸다. 송신 중 새 수신이 들어올 때 저장할 FIFO가 없으므로 연속 입력 무손실을 보장하지 않는다. 실제 보드에서는 한 문자씩 확인한 후 연속 입력을 별도 시험한다.

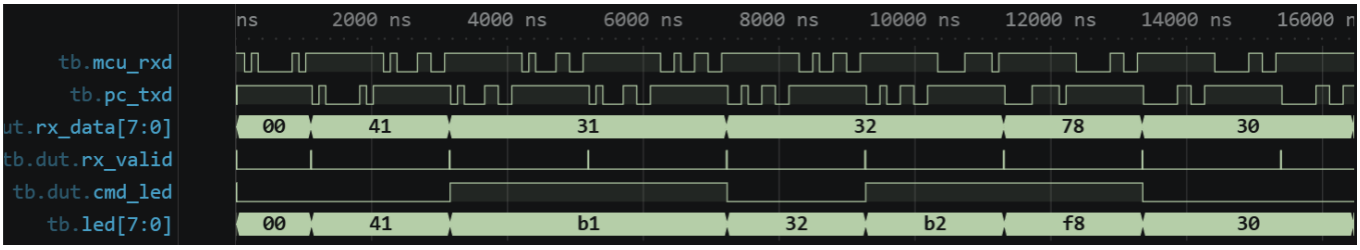


그림 7. 8개 입력에 따른 수신 바이트, 수신 유효 신호와 명령 LED의 전체 변화.

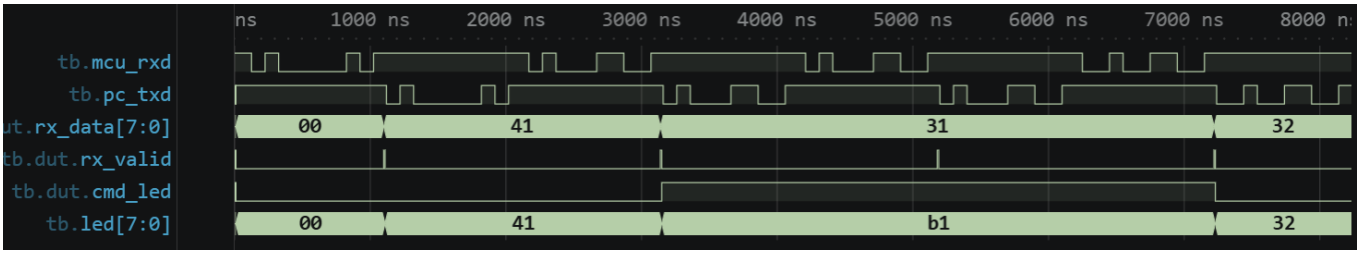


그림 8. A→1→1→2 상세 구간. 반복 켜기 명령과 반전 명령의 차이가 보인다.

5 STM32 UART 과제 5와 6

과제 5 PC와 MCU 입출력

UART2로 PC 입력을 받아 문자와 바이트 값을 출력한다. getchar는 입력이 들어올 때까지 대기하며, EOF 검사 뒤 uint8_t로 변환한다. 과제별 코드는 polling_routine에서 실행한다[2][4].

```
int ch = getchar();
if(ch != EOF) {
uint8_t data = (uint8_t)ch;
printf("%c / 0x%02X\r\n",
data, (unsigned int)data);
}
```

핵심 흐름을 요약한 코드이다. 실제 파일은 인쇄 가능 문자와 제어 문자를 구분한다. 터미널의 줄바꿈을 None으로 하고 "1", "A", "a"를 보내 0x31, 0x41, 0x61과 대응되는지 확인한다.

현재 확보한 빌드 근거

그림 9는 Build finished with exit code 0과 ninja: no work to do를 보여 준다. 빌드 명령이 정상 종료된 증거이며, 이 실행에서 전체 소스를 새로 컴파일했다는 의미는 아니다.

화면에는 실제 수신 문자나 시리얼 출력이 없으므로 과제 5의 보드 통신 검증은 실험 때 수행한다.

과제 6 PC에서 FPGA로 전달

PC→UART2→STM32→UART3→FPGA→FPGA USB-UART→PC 경로를 검증한다. STM32는 입력 바이트를 변경하지 않고 UART3으로 전달한다[2][4].

```
uint8_t data = (uint8_t)ch;
HAL_StatusTypeDef result =
HAL_UART_Transmit(
&huart3, &data, 1U, 100U);
```

통신 양쪽을 9600·8N1로 설정하고 MCU VCP와 FPGA USB-UART의 서로 다른 COM 포트를 연다. "A"를 입력하면 MCU 전송값 0x41과 FPGA 포트의 A를 비교한다.

HAL_OK인 0은 MCU 송신 완료를 나타낼 뿐 FPGA 수신까지 확인한 응답은 아니다. FPGA 포트에서 같은 바이트가 나타나야 전체 경로의 성공으로 판정한다.

배선과 점검 순서

STM32 PC10(TX)→Combo PORT A BDIO00(F8), GND↔GND를 연결하고 PORT A ENABLE을 켠다. 이 과제의 단방향 전달에는 PC11 응답선이 필수는 아니다.

출력이 없으면 COM 포트, baud, 공통 GND, TX→RX 방향, PORT ENABLE, FPGA 다운로드 순서로 점검한다. 그림 10 역시 빌드 완료 화면이며 실물 수신 증거와 구분한다.

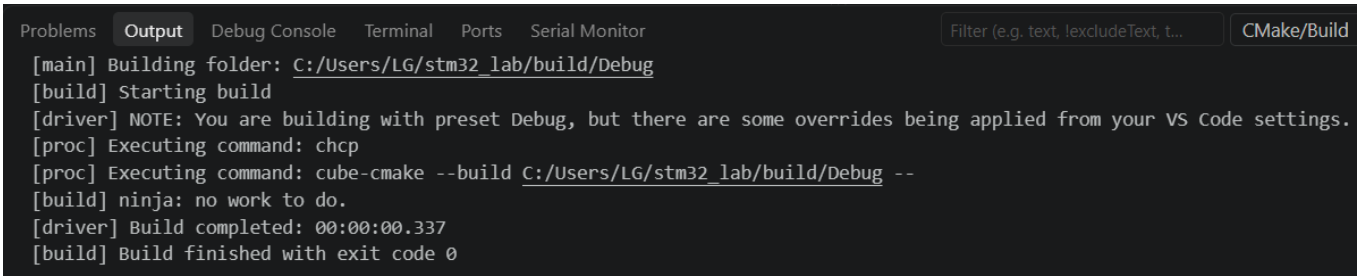


그림 9. 과제 5 빌드 출력 부분. 증분 빌드의 정상 종료를 확인했다.

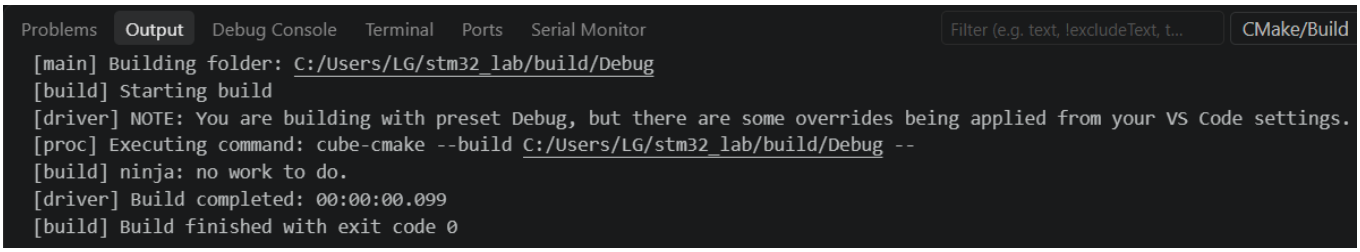


그림 10. 과제 6 빌드 출력 부분. 시리얼 수신값은 실험에서 별도로 확인한다.

6 STM32 과제 7과 통합 실험 계획

과제 7 UART 명령으로 LED 제어

STM32는 과제 6과 동일하게 명령 문자를 그대로 전달하고, FPGA가 명령을 해석한다. MCU 화면에 출력되는 명령 설명은 전송한 의도이며 실제 LED 상태의 회신이 아니다[4].

```
const char *meaning =
(data == '0') ? "OFF" :
(data == '1') ? "ON" :
(data == '2') ? "TOGGLE" :
"HOLD";
```

위 코드는 원본의 한글 설명 문자열을 영문으로 줄인 핵심 표현이다. 전송은 HAL_UART_Transmit(&huart3, &data, 1U, 100U)를 사용한다.

리셋 후 "1, 1, 2, 2, x, 0"을 차례로 보내 LED1이 켜짐→켜짐→꺼짐→켜짐→켜짐→꺼짐인지 기록한다. 각 단계의 PC 에코와 LED 사진을 함께 남긴다.

빌드 확인의 한계

그림 11은 정상 빌드 종료와 no work to do를 보여 준다. 코드 옆 중단점 표시는 그 줄에서 실행이 멈췄거나 Watch 값을 확인했다는 증거가 아니다. LED 실측 결과는 아직 포함하지 않는다.

실험실 수행 순서

① 배선 전 전원을 끄고 보드 핀·공통 GND·I/O 전압을 대조한다. STM32 신호선을 헤더의 +5V 핀에 연결하지 않는다.

② 서보·SRAM·DAC 프로젝트의 top과 XDC를 확인한 후 해당 bit 파일을 다운로드한다. 서보 펄스 폭, SRAM 주소별 값, DAC 주기를 순서대로 확인한다.

③ STM32 과제별 variants 파일을 수업 프로젝트의 ece2_474_routine.c에 적용하고 Build 후 다운로드한다. UART2와 UART3의 9600·8N1 설정을 확인한다.

④ 과제 5로 PC↔MCU를 먼저 확인하고, 과제 6에서 FPGA 에코를 비교한 다음 과제 7의 명령 순서를 시험한다.

⑤ 문제 발생 시 입력 조건·파형·핀 설정·수정 내용과 재검증 결과를 남긴다. 시뮬레이션 PASS, 빌드 성공, 실제 보드 동작을 각각 구분해 실험 후 보고서에 기록한다.

사전 점검에서 확인한 설계 사항

서보 소스는 50 MHz에서 pulse_us×50으로 클록 수를 계산한다. 기존 주석과 준비 자료에는 곱셈·나눗셈 경로의 타이밍 개선을 위해 바꾼 것으로 기록되어 있다. 이번 보고서에서는 구현을 다시 실행하지 않았다[4].

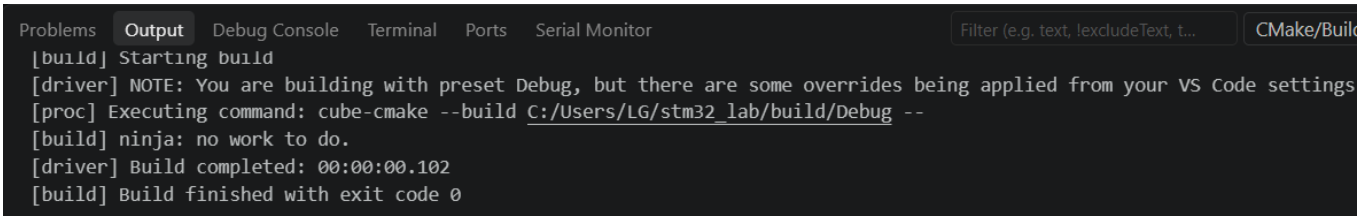


그림 11. 과제 7 빌드 출력 부분. 실물 LED 명령의 성공 여부는 보드에서 확인할 항목이다.

공통 판정 원칙

예상값과 실제 관측값은 같은 조건에서 비교한다. 축소된 TB의 ns 단위를 보드의 ms로 그대로 읽지 않고, 빌드 성공을 통신 또는 아날로그 동작 성공으로 확대 해석하지 않는다.

7 사전 검증 정리와 참고문헌

확인한 결과

대상	근거와 판정
서보	기존 로그 PASS 폭·포화·리셋
SRAM	기존 로그 PASS 16주소·Z·동시 요청
DAC	기존 로그 PASS 260회·순환·채널
UART LED	기존 로그 PASS 8바이트·명령·리셋
STM32 5·6·7	제공 화면 exit code 0 보드 통신은 확인 예정

사전 자료에서는 설정한 입력과 디지털 논리의 대응을 확인했다. 실험 당일에는 서보의 실제 각도와 펄스, SRAM의 실물 접근, DAC의 출력 전압 및 PC·MCU·FPGA 전체 통신 경로를 검증한다.

사진 원본과 재현 자료

그림 1~11은 제공 DOCX의 image1.png~image11.png에 순서대로 대응한다. 본문에는 UI 여백을 자른 파형·빌드 영역을 사용하고 원본 픽셀은 별도 보관한다.

저장소 루트에서 사진 11장을 바로 볼 수 있다. LAB06_사진과소스.zip 안에는 각 lab06_*의 src·sim·constraints·evidence와 stm32/variants가 있다. evidence에는 기존 build/sim의 로그·VCD를 보존했다. manifest.json에 파일 목록과 SHA-256을 기록했다.

소스와 제출 버전

저장소: github.com/josky-maker/LAB06
제출 태그: prelab-v1.0.0
소스 기준 커밋: 6ae8432d316a7851079364798afb2d6bd5e677c3

위 커밋은 사진 원본 11장과 소스·검증 자료 ZIP을 고정한 버전이다. PDF는 후속 커밋으로 추가하며 제출 태그는 PDF까지 포함한 버전을 가리킨다. 태그의 최종 커밋은 저장소에서 확인한다.

참고문헌

[1] M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 5th ed., Pearson, 2013. ISBN 9780132774208. 5~7장: 순차논리, 레지스터·카운터, 메모리. 출판사 제공 서지·목차: pearsonhighered.com/assets/preface/0/1/3/2/0132774208.pdf. 확인일 2026.10.21.

[2] 이해리, 「전자전기컴퓨터설계실험II STM32 실습 안내」, 2026.10.06, 6·10·12쪽. 제공 파일 STM32_4week_plan.pdf. LAB06 범위와 보고서 요건.

[3] 이해리, 「LAB 0 오리엔테이션」, 2026.09.07, 32·35~37쪽. 제공 파일 03.LAB0_OT_0907.pdf. 2단 PDF, 소스 버전 및 인용 기준.

[4] LAB06 로컬 실습 자료, lab06_servo·lab06_sram·lab06_dac·lab06_uart_led의 src 및 sim/tb_design.v, build/sim 아래 simulation.log; stm32/variants 과제 5·6·7 C 파일. 「lab06 사전보고서 사진.docx」 및 「LAB06_준비자료.md」. 확인일 2026.10.21. 실습 증거이며 외부 이론 출처와 구분.

[5] 조하늘, 「LAB05 실험 전 보고서」, 2026.10.13; 「example_project_pre_lab.pdf」. 표지와 2단 본문·그림 설명 형식 참고. LAB05의 실험 결과는 LAB06 결과로 전용하지 않았다.

본문의 일반 이론은 교과서 [1], 실험 범위는 [2], 구체적 구현과 관측 결과는 [4]에 근거한다. 문헌 번호와 본문 표시를 대응시켰다[3].